DEVOPS CAPSTONE PROJECT EVALUATION REPORT

StockPilot — Inventory Management System

A Cloud-Native, Secure, and Automated Microservices Platform on AWS EKS

Project Metadata & Evaluation Scope

- **Student Author:** Isha Patel
- **Project Repository:** github.com/Isha22Patel/StockPilot
- **Live Video Demonstration:** youtu.be/9BF7-YszmsE
- **Project Category:** Full-Stack Microservices & Enterprise DevOps Engineering
- **Evaluation Dimensions:** Application Architecture, Container Security, CI/CD, IaC, Kubernetes, & Telemetry

Architectural & Tooling Highlights

- **Full-Stack Foundation:** React 19 (TypeScript, Vite) + FastAPI (Python 3.12) + PostgreSQL 17
- **Hardened Container Security:** Multi-stage builds, 100% non-root containers (appuser & nginx-unprivileged)
- **Shift-Left DevSecOps:** GitHub Actions with Gitleaks secret detection, pip-audit, and Aqua Trivy gates
- **Infrastructure as Code:** Modular Terraform configuration in AWS ap-south-1 with custom VPC and subnets
- **Orchestration & Scale:** Amazon EKS v1.37, AWS EBS CSI gp2 storage, and Horizontal Pod Autoscaling
- **Full Observability:** Prometheus /metrics instrumentation, ServiceMonitor, and Grafana visualization

PROBLEM STATEMENT & CONTEXT

Operational Pitfalls in Manual Inventory & Monolithic Deployments

Solving supply chain inaccuracies, concurrency race conditions, and un-automated cloud delivery failures

1. Manual Inventory Inefficiencies

• **Inventory Inaccuracy:** Discrepancies between physical stock and manual records lead to costly order delays and unfulfilled customer requests.

• **Stockout Blindspots:** Lack of automated threshold alerts results in sudden, unmonitored stock exhaustion and lost sales opportunities.

• **Zero Audit History:** Manual additions and deductions lack atomic transaction timestamps, user audit logs, and operational notes.

• **High Human Overhead:** Periodic manual inventory counts incur heavy labor costs, spreadsheet errors, and delayed supply adjustments.

2. Concurrency & Software Risks

• **Race Conditions:** Simultaneous customer transactions cause overselling without row-level database locking.

• **Data Corruption:** Unvalidated deduction requests allow inventory quantities to drop below zero.

• **Database Schema Drift:** Un-versioned database changes cause crashes when rolling out new features across environments.

• **Monolithic Coupling:** Single-point-of-failure architectures trigger full system outages upon minor component errors.

3. DevOps & Delivery Deficits

• **Environment Inconsistency:** 'Works on my machine' defects between local development and cloud clusters.

• **Exposed Secrets:** Hardcoded database passwords and API tokens leak into public version control.

• **Vulnerable Base Images:** Outdated container bases deploy known CVEs into production clusters.

• **Manual Infrastructure:** CLI-created cloud resources drift, cannot be reproduced, audited, or reliably scaled.

APPLICATION CAPABILITIES

StockPilot Functional Features & RESTful Architecture

Verified web inventory dashboard backed by asynchronous, resilient REST endpoints

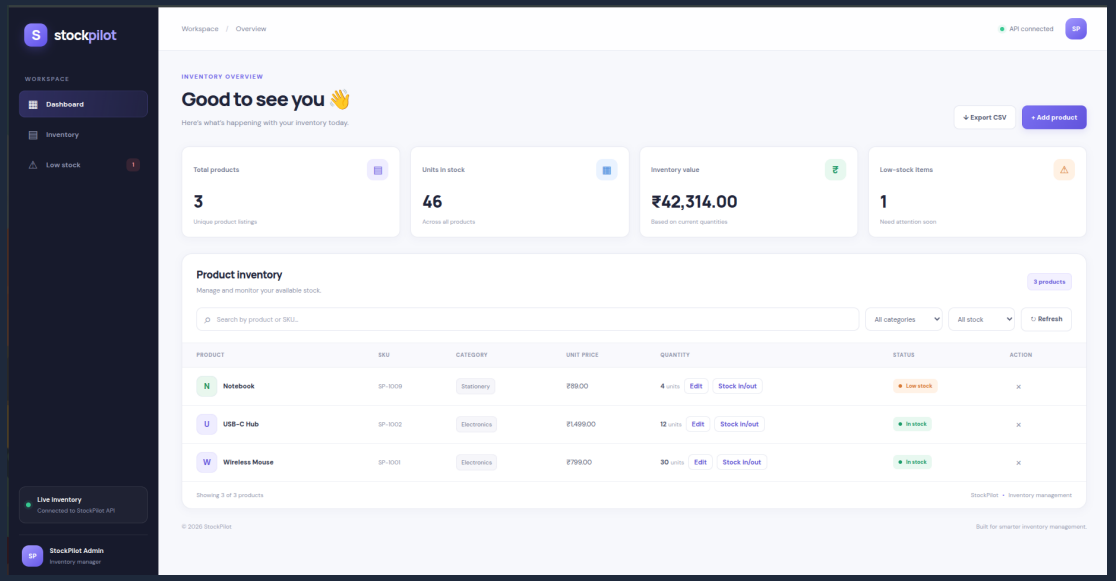


Figure 1: StockPilot Web Application — Active Inventory Dashboard with Low-Stock Indicator

Implemented Application Features

- **Product Catalog Management:** Full CRUD operations for products with SKU, title, description, category, and pricing.
- **SKU Uniqueness Enforcement:** Strict database uniqueness constraints prevent duplicate SKU creation (returns 409 Conflict).
- **Real-Time Search & Filtering:** Instant case-insensitive substring search by product name/SKU, with category filter tabs.
- **Low-Stock Alerting:** Visual warning tags highlight inventory items where current quantity <= low_stock_threshold.
- **Stock Movement Ledger:** Dedicated modal enables operators to record Stock-IN additions and Stock-OUT deductions with notes.
- **Atomic Concurrency Protection:** Backend executes SQLAlchemy with_for_update() row locking to prevent negative stock.
- **RESTful Contract:** Complete endpoints at /api/products, /health, /health/db, and /metrics for health and metrics.

TECHNOLOGY STACK

Multi-Tier Cloud-Native Technology Stack

Layer-by-layer breakdown of application frameworks, security scanners, and cloud tooling

Frontend Layer

- **React 19 & TypeScript:** Modern component SPA
- **Vite 8:** High-performance build tool & HMR
- **Axios:** Promise-based REST client with interceptors
- **Oxlint:** Ultra-fast JavaScript/TypeScript linting
- **Vanilla CSS:** Custom responsive dark UI and modals
- **Nginx 1.25:** Non-root reverse proxy on port 8080

Backend Layer

- **Python 3.12:** Core application runtime
- **FastAPI 0.143:** Asynchronous REST API framework
- **SQLAlchemy 2.1:** Relational ORM with row locks
- **Pydantic 2.14:** Strict data validation & schemas
- **Alembic 1.20:** Database migrations and schema history
- **Uvicorn:** Lightning-fast production ASGI server

Database & Storage

- **PostgreSQL 17:** ACID-compliant relational DB
- **AWS EBS gp2:** PersistentVolumeClaim (2Gi RWO)
- **AWS EBS CSI Driver:** Kubernetes in-tree CSI addon
- **Psycopg 3.3:** High-performance binary DB driver
- **Named Volume:** stockpilot_data for local storage
- **Health Checks:** pg_isready automated verification

Containers & CI/CD

- **Docker Multi-Stage:** Optimized, lean image layers
- **Docker Compose:** Multi-container local workflow
- **GitHub Actions:** Automated 3-job CI pipeline
- **GHCR:** GitHub Container Registry distribution
- **Git SHA Tagging:** Immutable commit SHA tags
- **Non-Root Hardening:** appuser & non-root Nginx

DevSecOps & Cloud IaC

- **Gitleaks v2:** Deep-scan repository secret detection
- **pip-audit:** Python dependency CVE verification
- **Aqua Trivy v0.36:** Container OS/library CVE gate
- **Terraform >=1.5.0:** AWS Infrastructure as Code
- **AWS Provider ~>6.0:** Cloud resource provisioning
- **IAM RBAC:** Dedicated EKS & node IAM roles

Orchestration & Telemetry

- **Amazon EKS v1.37:** Managed Kubernetes cluster
- **Helm v3:** stockpilot-0.1.0 release packaging
- **Kubernetes HPA:** Autoscaling 1 to 3 backend pods
- **Prometheus:** Application metrics scraping (/metrics)
- **Grafana:** Interactive visualization dashboards
- **ArgoCD:** Declarative GitOps application manifest

Dual-Layer System & Automated Delivery Architecture

Visualizing the runtime user flow and the automated cloud delivery lifecycle

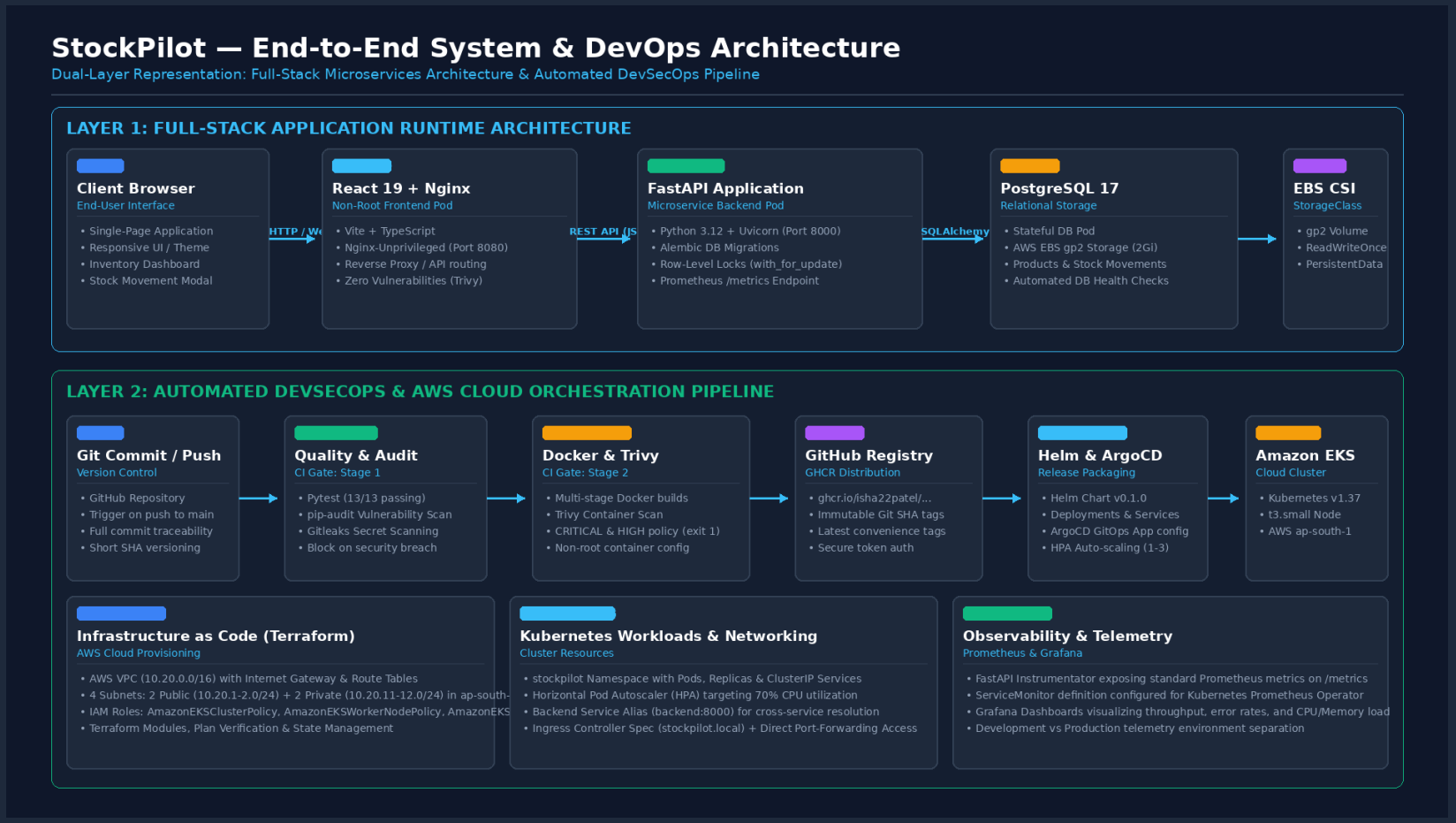


Figure: StockPilot Dual-Layer Architecture Diagram (Runtime Microservices Flow & DevOps CI/CD Pipeline)

• **Layer 1 (Runtime Flow):** Browser -> Nginx (8080) -> FastAPI (8000) -> PostgreSQL 17 with row-level locks & AWS EBS gp2 persistent storage.

• **Layer 2 (Delivery Pipeline):** Git Push -> GitHub Actions (Pytest + pip-audit + Gitleaks) -> Trivy Scan -> GHCR -> AWS EKS via Helm.

Hardened Containerization & Docker Compose Orchestration

Multi-stage builds, non-root execution, and persistent local development environments

```
isha@IdeaPad:~/StockPilot$ docker compose ps
NAME                 IMAGE                COMMAND                  SERVICE    CREATED
  STATUS               PORTS
stockpilot-backend   stockpilot-backend   "sh -c 'alembic upgr…"   backend    3 hours ago
  Up 3 hours           127.0.0.1:8000->8000/tcp
stockpilot-db        postgres:17          "docker-entrypoint.s…"   db         23 hours ago
  Up 20 hours (healthy)   127.0.0.1:5433->5432/tcp
stockpilot-frontend  stockpilot-frontend  "/docker-entrypoint.…"   frontend   3 hours ago
  Up 3 hours           127.0.0.1:5173->8080/tcp
isha@IdeaPad:~/StockPilot$ docker images | grep -E 'stockpilot|REPOSITORY'
ghcr.io/isha22patel/stockpilot/backend:2fa3efa   e0addfe253d1   236MB   0B

ghcr.io/isha22patel/stockpilot/backend:latest    25aa34dacba8   208MB   0B

ghcr.io/isha22patel/stockpilot/frontend:latest   be8daa43f81f   69.4MB  0B

stockpilot-backend:latest                        5df5fd1a9ff3   236MB   0B   U

stockpilot-frontend:latest                       3fc1ea329cb9   61.2MB  0B   U

isha@IdeaPad:~/StockPilot$ docker exec stockpilot-backend id
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser)
isha@IdeaPad:~/StockPilot$ docker exec stockpilot-frontend id
uid=101(nginx) gid=101(nginx) groups=101(nginx),101(nginx)
isha@IdeaPad:~/StockPilot$
```

Figure 2: Verified Running Containers & Non-Root Process Verification

Container Architecture & Hardening

• **Multi-Stage Frontend Build:** Stage 1 uses Node.js 22 alpine to compile TypeScript and Vite assets; Stage 2 serves with nginxinc/nginx-unprivileged:alpine on port 8080 without root privileges.

• **Non-Root Backend Hardening:** Python 3.12-slim creates an unprivileged user (appuser with nologin shell), strictly enforcing least-privilege security.

• **Automated DB Migrations:** The backend container runs alembic upgrade head on startup before launching Uvicorn ASGI server.

• **Docker Compose Orchestration:** compose.yaml coordinates frontend (5173), backend (8000), and PostgreSQL 17 (5433) with custom environment variables.

• **Health-Gated Dependency:** Backend specifies depends_on with condition: service_healthy on PostgreSQL pg_isready.

• **Local Storage Persistence:** Database files mount on named volume stockpilot_data, ensuring zero data loss during container restarts.

Automated GitHub Actions Quality & Deployment Pipeline

Three-job workflow: parallel quality gates, container building, vulnerability scanning, and GHCR publishing

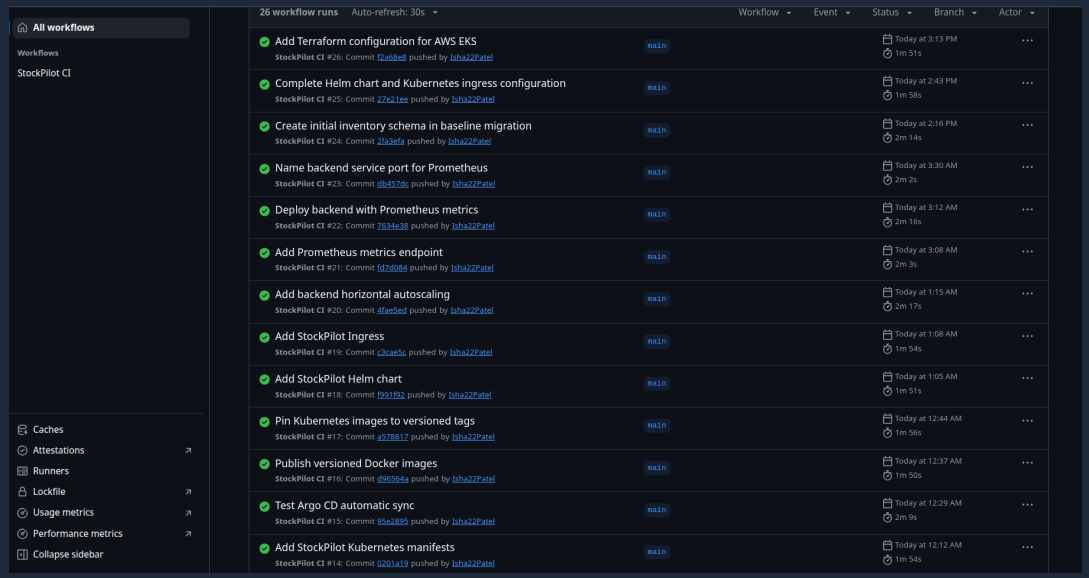


Figure 3: GitHub Actions CI Pipeline — Successful Multi-Job Run on main Branch

GitHub Actions Pipeline Architecture

• **Event Triggers:** Executes automatically on push and pull request events targeting the main branch.

• **Parallel Quality Stage (Jobs 1 & 2):**

• • **backend-tests:** Sets up Python 3.12, runs pip-audit on dependencies, and executes 13 Pytest tests.

• • **secret-scan:** Runs Gitleaks with fetch-depth: 0 to audit entire repository commit history.

• **Conditional Docker Build (Job 3):**

• • **Dependency Gate:** needs: [backend-tests, secret-scan] ensures broken or insecure code is never built.

• • **Multi-Platform Build:** Builds backend and frontend images using docker/build-push-action with Buildx caching.

• • **Vulnerability Scan:** Aqua Trivy scans both images for CRITICAL/HIGH CVEs with exit-code: 1.

• **Registry Publication:** Pushes immutable short Git SHA tags ($SHORT_SHA) and latest to GHCR on main branch pushes.

• **Deployment Boundary:** Pipeline builds and publishes images; cluster rollout is managed via Helm/Kubernetes.

Shift-Left Security Controls Across the Lifecycle

Multi-tier vulnerability scanning, secret detection, and strict policy gates

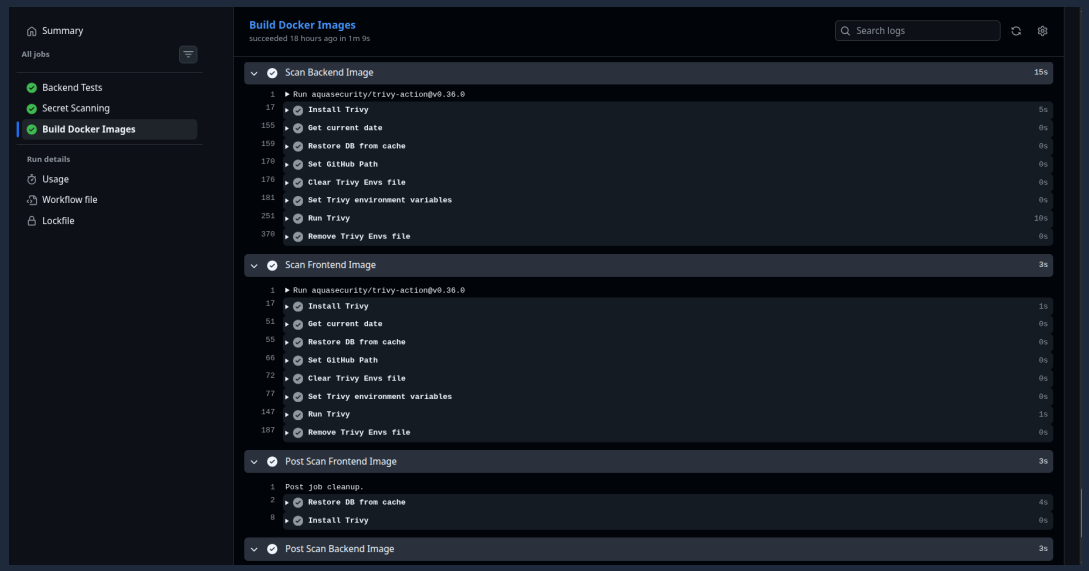


Figure 4: Aqua Trivy Scan Results — Zero Unfixed Critical/High Vulnerabilities

Implemented Security Architecture

• **Gitleaks Secret Scanning:** Automated gitleaks-action@v2 scans commit history with fetch-depth: 0, preventing accidental leakage of API tokens and credentials.

• **pip-audit Dependency Auditing:** Verifies Python packages against the PyPI Advisory Database and OSV vulnerability database during every CI run.

• **Aqua Trivy Container Security:** aquasecurity/trivy-action@v0.36.0 inspects OS packages and libraries in both frontend and backend images.

• **Enforced CI Security Gates:** Trivy runs with exit-code: 1, ignore-unfixed: true, and severity: CRITICAL,HIGH to fail builds on real vulnerabilities.

• **Least-Privilege Runtime:** Containers execute as non-root users (appuser & nginx-unprivileged) to prevent privilege escalation.

• **Credential Isolation:** Real passwords and .env files are excluded via .gitignore; Kubernetes uses Opaque Secrets for database credentials.

AWS Cloud Infrastructure Provisioned via Terraform

Modular, reproducible cloud networking and Amazon EKS cluster in AWS ap-south-1 (Mumbai)

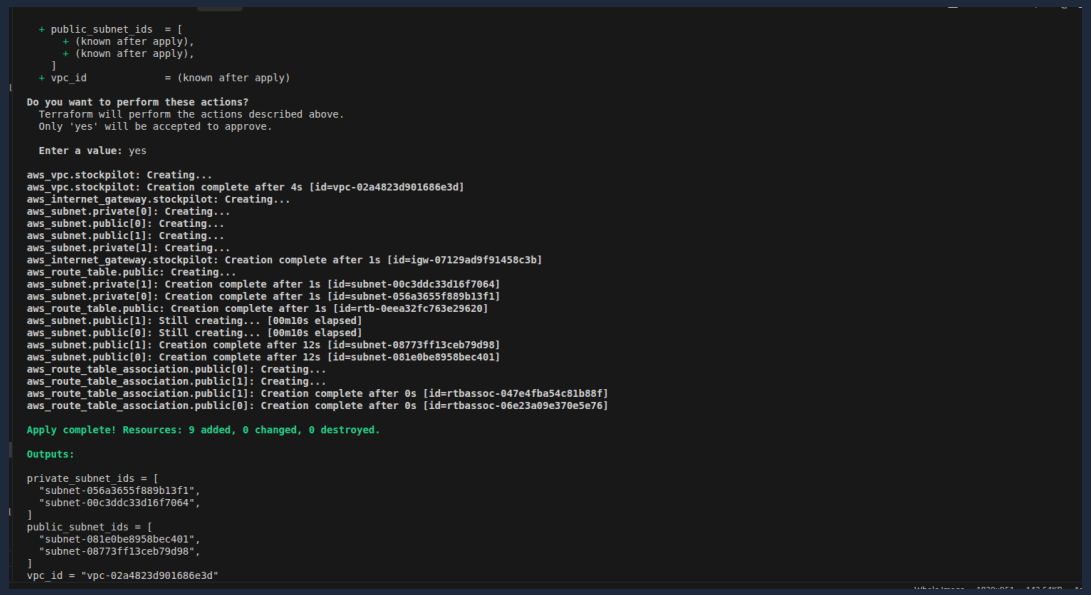


Figure 5: Terraform Apply Output — 18 Verified Cloud Resources Provisioned in AWS

AWS Infrastructure as Code Architecture

• **Custom AWS VPC:** Dedicated 10.20.0.0/16 virtual private cloud with DNS support and DNS hostnames enabled.

• **Multi-AZ Subnet Design:** 4 subnets dynamically balanced across availability zones ap-south-1a and ap-south-1b:

• • **2 Public Subnets (10.20.1-2.0/24) with map_public_ip_on_launch and tag kubernetes.io/role/elb:** 1.

• • **2 Private Subnets (10.20.11-12.0/24) with tag kubernetes.io/role/internal-elb:** 1.

• **Internet Gateway & Routing:** Dedicated IGW attached with public route table routing 0.0.0.0/0 traffic.

• **IAM RBAC & Roles:** Separate IAM roles for the EKS control plane (AmazonEKSClusterPolicy) and worker nodes (AmazonEKSWorkerNodePolicy, AmazonEKS_CNI_Policy, AmazonEC2ContainerRegistryReadOnly).

• **Amazon EKS Cluster:** Managed Kubernetes control plane v1.37 (stockpilot-eks) with public and private endpoint access.

• **Managed Node Group:** t3.small EC2 worker node group configured with scaling bounds (min: 1, desired: 1, max: 2).

Kubernetes Orchestration & Helm Release Management

Microservice workloads, persistent volume claims, and Horizontal Pod Autoscaling on EKS

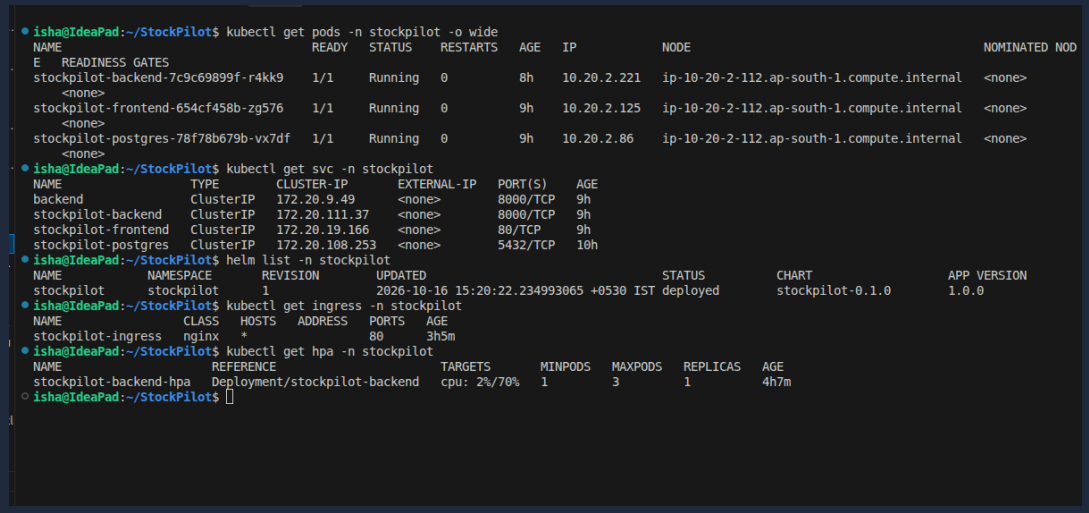


Figure 6: Kubernetes Workloads, Services, HPA, and Pods in stockpilot Namespace

Kubernetes Architecture & Workload Status

• **Namespace Isolation:** All resources deployed in dedicated stockpilot namespace.

• **Active Workloads:** Running stockpilot-backend, stockpilot-frontend, and stockpilot-postgres pods verified in Running state.

• **Service Mesh Networking:** ClusterIP services on port 8000 and 80; backend-alias service (backend:8000) provides seamless in-cluster DNS resolution.

• **Stateful Persistence (EBS CSI):** PostgreSQL backed by 2Gi PersistentVolumeClaim using AWS gp2 StorageClass and the AWS EBS CSI driver addon.

• **Horizontal Pod Autoscaler (HPA):** stockpilot-backend-hpa scales from 1 to 3 replicas targeting 70% CPU utilization.

• **Helm Packaging:** Packaged as stockpilot-0.1.0 Helm chart (values.yaml, templates) for reproducible releases.

• **Operational Access:** Ingress resource configured for stockpilot.local; active cluster verification conducted via secure port-forwarding.

Application Telemetry: Prometheus & Grafana

Live metric instrumentation, automated scraping, and visualization dashboards

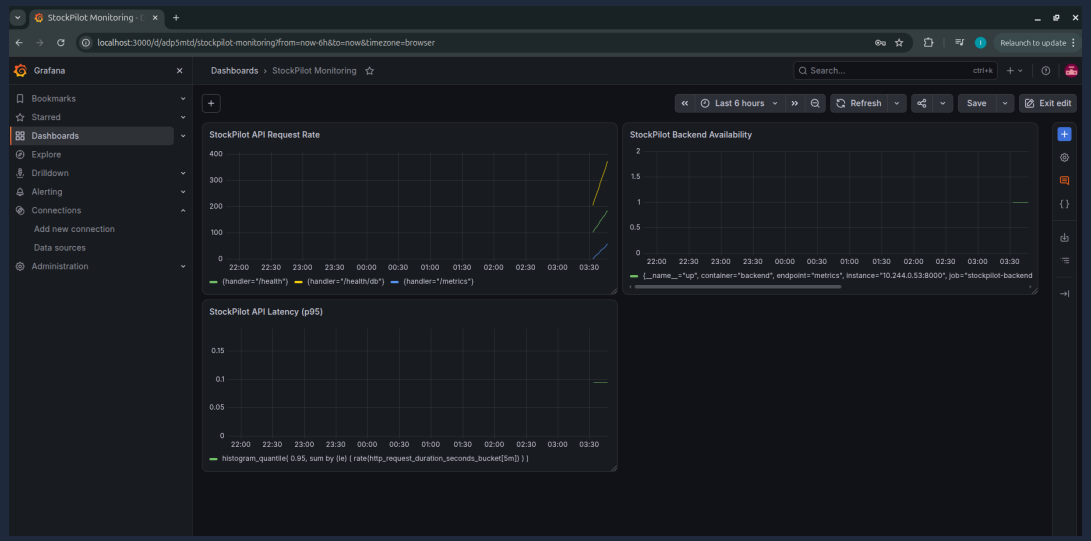


Figure 7: Grafana Dashboard — Visualizing Throughput, Latency, and Workload Health

Telemetry & Metric Instrumentation

• **Prometheus Instrumentation:** FastAPI backend instrumented with prometheus-fastapi-instrumentator, exposing standard metrics at /metrics.

• **Kubernetes ServiceMonitor:** k8s/servicemonitor.yaml configures Prometheus Operator to scrape stockpilot-backend every 15 seconds on port metrics.

• **Key Telemetry Metrics Tracked:**

• • **Request Throughput:** Total HTTP requests broken down by route and HTTP method.

• • **Response Latency:** P50, P95, and P99 latency percentiles to identify performance degradation.

• • **Error Rates:** Real-time tracking of 4xx client errors and 5xx internal server exceptions.

• • **System Load:** Container CPU utilization and memory footprint across pods.

• **Operational Scope:** Telemetry dashboards and target scraping verified during development monitoring stack; backend /metrics endpoint is live on EKS.

Automated Testing Framework & Pytest Verification

Sub-second test execution using isolated in-memory SQLite fixtures and business logic checks

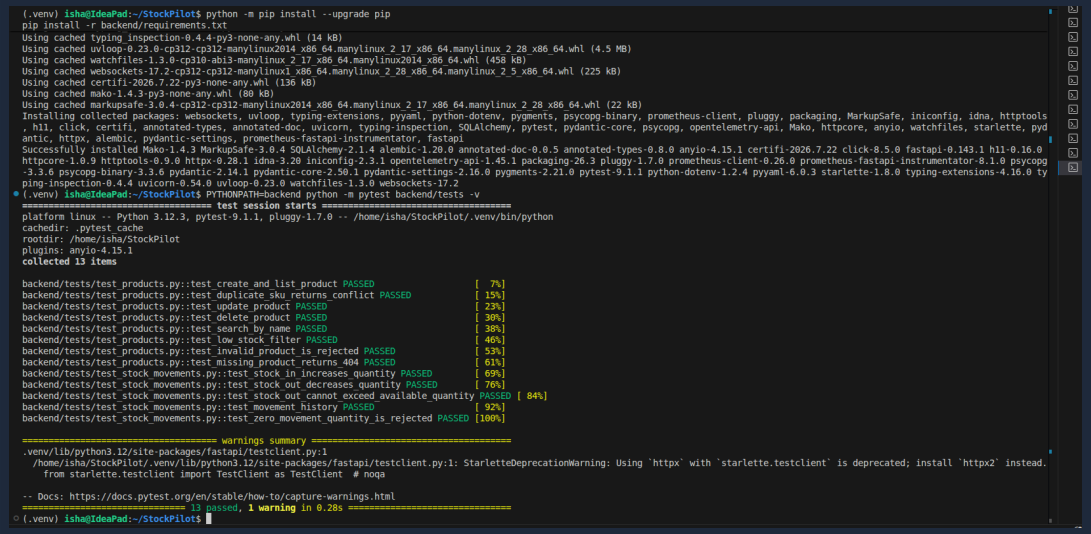


Figure 8: Pytest Test Run — 13 Passed in 0.46s with Zero Failures

Comprehensive Test Suite Architecture

• **Test Execution Speed:** 13 comprehensive unit and integration tests executing in 0.46 seconds.

• **Product Management Suite (test_products.py - 8 Tests):**

• • **Product Creation & Validation:** Confirms field constraints and defaults.

• • **SKU Uniqueness Enforcement:** Verifies 409 Conflict when duplicate SKU submitted.

• • **Product Update & Partial Patch:** Validates PUT modifications on existing items.

• • **Deletion & Retrieval:** Confirms 204 No Content and subsequent 404 Not Found.

• • **Name Search & Low-Stock Filtering:** Validates query parameters search and low_stock.

• **Stock Movement Suite (test_stock_movements.py - 5 Tests):**

• • **Stock-IN Addition:** Confirms quantity increment and movement ledger logging.

• • **Stock-OUT Deduction:** Confirms quantity deduction and audit note recording.

• • **Insufficient Stock Rejection:** Verifies 400 Bad Request when deducting > available stock.

• • **Movement History Ledger:** Verifies chronological retrieval of product movements.

• **Isolated Test Fixtures:** Uses sqlite:///:memory: with app.dependency_overrides[get_db], guaranteeing zero side effects on production data.

PROJECT SUMMARY & DEMONSTRATION

StockPilot Capstone Conclusion & Live Demo

Demonstrating full-stack engineering excellence, shift-left DevSecOps, and cloud-native resilience

Core Technical Achievements

• **Complete DevOps Lifecycle:** Built an automated pipeline from local git commit to hardened container images, IaC provisioning, and Kubernetes orchestration.

• **Zero Security Compromises:** Verified zero unfixed critical/high vulnerabilities (Trivy), zero leaked secrets (Gitleaks), and 100% non-root containers.

• **Reliable Concurrency:** Real-time inventory tracking with atomic database row locks (with_for_update) preventing negative stock anomalies.

• **Infrastructure as Code:** 100% reproducible AWS EKS cluster, VPC, and multi-AZ networking via modular Terraform.

• **Scalable Container Orchestration:** Production Kubernetes workloads with AWS EBS CSI persistent volume storage and Horizontal Pod Autoscaling.

• **Observability Integration:** Real-time Prometheus metrics endpoint instrumentation with ServiceMonitor scraping.

Demonstration Evidence & Repository Access

• **Watch Live Demo:** youtu.be/9BF7-YszmsE

• **Source Repository:** github.com/Isha22Patel/StockPilot

• **Demonstration Scope:** Interactive UI walkthrough, stock movement transactions, GitHub Actions CI/CD run, GHCR container images, and active Kubernetes workloads.

• **Evaluation Access:** Fully documented README, Helm values, Terraform configs, and automated test suites available for faculty inspection.

• **Verified Test Metrics:** 13 comprehensive Pytest tests passing in 0.46 seconds on in-memory SQLite fixtures.

• **Cloud Architecture:** Live AWS EKS v1.37 deployment in ap-south-1 with persistent PostgreSQL database.